Maqwad — Backend API Requirements

Provider & Admin Dashboards — for Backend Team

Date: June 28, 2026

Frontend stack: React 19 + TypeScript (strict)

Target backend: .NET (Miqwad API)

Source: src/modules/*/api/*.ts | src/modules/*/types.ts | src/shared/mocks/handlers/*.ts

Rule: Every field name, type, and path is quoted directly from the frontend source — nothing is invented.

This document lists every HTTP endpoint the Maqwad provider & admin dashboards call (or will call). The **Missing-Endpoint Checklist** and **Open Questions** sections are the key deliverables for the backend team. Approximately **84% of required endpoints are not yet implemented** in the current .NET backend.

Maqwad — Backend API Requirements (Provider & Admin Dashboards)

> **Derived from:** `src/modules/*/api/*.ts`, `src/modules/*/types.ts`, `src/shared/mocks/handlers/*.ts` **Date:** 2026-06-28 | **Frontend stack:** React 19 + TypeScript (strict) | **Target backend:** .NET (Miqwad API) **Rule:** Every field name, type, and path in this document is quoted directly from the frontend source — nothing is invented.

---

1. Executive Summary

The existing .NET backend (`miquad-api`) is **customer-facing**. It covers the consumer app (Vehicles, Lookups, Orders, Products/storefront, Appointments, TowTrips, Reviews, Conversations) and a minimal provider-approval surface (approve/reject/suspend). The provider dashboards (Dealer, Workshop, Scrap) and the full Admin panel are built on the frontend but have **zero backend support** today.

Area	Total Endpoints Needed	Currently EXISTS	Missing / Partial
Auth & Users	10	7 (EXISTS)	3 MISSING
Provider Onboarding	8	4 PARTIAL	4 MISSING
Dealer Dashboard	14	0	14 MISSING
Workshop Dashboard	4	0	4 MISSING
Scrap Dashboard	10	0	10 MISSING
Admin Panel	49	3 PARTIAL	46 MISSING
Discovery (customer)	5	0	5 MISSING
Services / Lookups	5	3 EXISTS	2 MISSING
TOTAL	**105**	**~17 (16%)**	**~88 (84%)**

Core gap: The backend is customer-facing. The 84% of missing endpoints are entirely provider- and admin-domain. The three provider types (dealer/workshop/scrap) each need their own namespaced endpoint family. The Scrap module introduces a net-new **Escrow** domain that does not exist anywhere in the current backend. The Admin panel requires ~49 endpoints spanning users, providers, subscriptions, revenues, ads, audit logs, complaints, notifications, and system settings.

---

2. Global / Cross-Cutting Requirements

2.1 Response Envelope

The frontend expects:

```
// List endpoints interface PaginatedResponse<T> { items: T[]; page: number; // 1-based pageSize: number; total: number; // total record count totalPages: number; // Math.ceil(total / pageSize) } // Single resource endpoints — return the object directly (no wrapper) // e.g. GET /dealer/products/{id} → Product (not { data: Product }) // Error envelope interface ApiErrorResponse { success: false; error: { code: string; // stable machine-readable code e.g. "NOT_FOUND" message: string; // human-readable fields?: Record<string, string[]>; // validation errors per field }; }
```

> **Decision needed:** The frontend currently expects either a bare `T` or `PaginatedResponse<T>`. If the backend wraps responses in `{ success: true, data: T }`, a response-interceptor adapter is required in `src/shared/lib/axios.ts`. **Recommended:** backend emits bare responses and uses HTTP status codes for error signalling.

2.2 Field Casing

- **Frontend:** camelCase fields + camelCase string enum values (`"active"`, `"dealer"`, `"monthly"`)

- **Current backend:** likely PascalCase (`"Active"`, `"Dealer"`, `"Monthly"`)

Options (DECISION NEEDED): 1. Backend emits camelCase JSON (via `JsonNamingPolicy.CamelCase` in .NET) — **recommended, no FE change needed** 2. Frontend adds a global Axios response interceptor to convert PascalCase → camelCase — adds maintenance burden

2.3 ID Strategy

- Frontend types use `id: string` (UUID) for all new domain entities (products, orders, subscriptions, etc.)
- Legacy provider IDs (`providerId`, `workshopId`, `scrapId`, `dealerId`) are `number` — this mirrors the existing Swagger where `ServiceProviders/{id}` uses an integer PK
- **Backend owns ID generation** — the frontend never mints IDs for persisted entities
- Mock-invented IDs to replace with real UUIDs: `"prod_001"`, `"ord_001"`, `"shp_001"`, `"sub_ws_001"`, `"pr_001"`, `"esc_001"`, `"cmp_001"`

2.4 Money

All monetary amounts are `number` (SAR). Currency is implicit (platform is SAR-only). No `currency` field is stored per record — the response contract assumes SAR.

2.5 Dates

All date/datetime fields are ISO-8601 strings (e.g., `"2026-06-28T14:30:00Z"`). Date-only fields (e.g., `registrationDate`) use `"YYYY-MM-DD"`.

2.6 Auth — OTP vs Password (CRITICAL RECONCILIATION NEEDED)

The frontend uses a **phone + OTP** flow with no password:

> `POST /auth/register { phoneNumber: string } → { verificationId, resendAfter } POST /auth/verify-otp { verificationId, code } → { accessToken, refreshToken, user }`

The existing backend login reportedly uses `emailOrPhone + password`. This is a **hard incompatibility**:

> **Question for backend team:** Is OTP login already implemented on `/auth/register` + `/auth/verify-otp`? If not, this is the single highest-priority auth change. The provider/admin dashboards are entirely OTP-gated; there is no password field in any frontend form or type.

All protected endpoints require `Authorization: Bearer <accessToken>`. Role enforcement: - `customer` — can access Discovery, Vehicles, Favorites - `provider` + `providerStatus: "approved"` — can access their type-specific provider dashboard - `admin` / `super_admin` — can access `/admin/*`

2.7 Provider Type Enum Mapping

The frontend's `ProviderType = "dealer" | "workshop" | "scrap"` must map to backend enum names:

Frontend value	Suggested backend enum	Notes
`"dealer"`	`PartsVendor` (existing?) or `Dealer`	Confirm with backend team
`"workshop"`	`Workshop`	Likely exists
`"scrap"`	`ScrapYard` or `Scrap`	Likely new

The `User.role` values `"customer" | "provider" | "driver" | "admin" | "super_admin"` must map to backend roles accordingly.

2.8 `WorkshopStats` — Note

The workshop dashboard (`WorkshopStats`) has only two KPIs today (`rating`, `activeServicesCount`) — conversations KPI is deferred. No dedicated stats endpoint was added to `workshopApi.ts` yet; the dashboard reads these from the profile. **However**, the scrap module has a dedicated `GET /scrap/stats` returning `ScrapStats`. The dealer module currently has no stats endpoint (the dashboard uses product/order counts from paginated responses). This is consistent with each module's current implementation.

---

3. Per-Module Endpoint Tables

3.1 Auth & Users

#	Method	Path	Purpose	Request (key fields)	Response (key fields)	Screen	Status
A1	POST	`/auth/registe`	Send OTP to phone	`phoneNumber: string`	`{ verificationId: string, resendAfter: number }`	Login / Register	EXISTS
A2	POST	`/auth/verify-`	Verify OTP, issue tokens	`{ verificationId, code }`	`{ accessToken, refreshToken, user: User }`	OTP Entry	EXISTS
A3	POST	`/auth/refresh`	Renew access token	`{ refreshToken }` (body or cookie)	`{ accessToken, refreshToken }`	Axios interceptor (silent)	EXISTS
A4	POST	`/auth/logout`	Invalidate session	—	`{ success: true }`	Any screen	EXISTS
A5	GET	`/users/me`	Get current user profile	—	`User`	App bootstrap / Nav	EXISTS
A6	PUT	`/users/me`	Update name / email / role	`{ fullName, email?, role }`	`User`	Profile complete step	EXISTS
A7	POST	`/users/me/avat`	Upload avatar image	`FormData: file` (multipart)	`User` (with updated `avatarUrl`)	Profile settings	EXISTS
A8	GET	`/onboarding/ac`	Get post-registration account info	—	`{ accountType, ownerName, email, phone }`	Onboarding step 1	MISSING
A9	POST	`/onboarding/dc`	Upload KYC docs during onboarding	`FormData: file` + doc kind	`{ fileId: string, fileName: string }`	Onboarding step 2	MISSING
A10	POST	`/onboarding/su`	Submit provider application for admin review	—	`{ submittedAt: string }`	Onboarding step 3	MISSING
A11	GET	`/onboarding/re`	Poll application review state	—	`ReviewTimeline` (array of `{ id, state: "done"|"active"|"pending" }`)	Pending approval page	MISSING

`User` type (all fields the frontend reads from this endpoint):

```
{ id: string; phoneNumber: string; fullName: string; email: string | null; role: "customer" | "provider" | "driver" | "admin" | "super_admin"; avatarUrl: string | null; isProfileComplete: boolean; providerId?: number | null; // set when role === "provider" providerStatus?: "pending" | "approved" | "rejected" | null; providerRejectionReason?: string | null; permissions?: string[]; // RBAC codes; super_admin gets ["*"] }
```

3.2 Providers / Onboarding

#	Method	Path	Purpose	Request (key fields)	Response (key fields)	Screen	Status
P1	POST	`/ServiceProvi`	Register new provider	`RegisterProvi` (see below)	`ProviderProfi`	Provider onboarding wizard	EXISTS
P2	GET	`/ServiceProvi`	Get provider profile by numeric ID	`fi` —	`ProviderProfi`	Admin review drawer; customer discovery	EXISTS
P3	POST	`/ServiceProvi`	Attach KYC documents to existing provider	`um` `{ documents: ProviderDocument[] }`	`ProviderProfi`	Onboarding doc upload	PARTIAL (not confirmed in Swagger)
P4	GET	`/provider/me`	Get the calling provider's own profile (no ID needed — derived from JWT)	—	`ProviderProfi`	Provider dashboard bootstrap	MISSING
P5	GET	`/providers/{i`	List services offered by provider	—	`ProviderServi`	Provider profile; customer discovery	EXISTS
P6	POST	`/providers/{i`	Add a service	`CreateProvide`	`ProviderServi`	Provider service management	PARTIAL (mocked)
P7	PUT	`/providers/{i`	Update a service	`se` `UpdateProvide`	`ProviderServi`	Provider service management	PARTIAL (mocked)
P8	DELETE	`/providers/{i`	Remove a service	`se` —	`void`	Provider service management	PARTIAL (mocked)

`RegisterProviderRequest`:

```
{ companyName: string; email: string; phone: string; password: string; // ■ Only field requiring reconciliation vs OTP flow lat?: number | null; lng?: number | null; address?: string; city?: string; workingHours?: string; categoryIds?: number[]; documents?: Array<{ type: "commercial"|"tax"|"identity", fileName: string, fileSize: number }>; }
```

`ProviderProfile` (all fields read by admin review and provider pages):

```
{ id: number; type: "dealer" | "workshop" | "scrap"; userId: string; companyName: string; email: string; phone: string; lat: number | null; lng: number | null; address: string | null; city: string | null; workingHours: string | null; rating: number; totalRatings: number; isVerified: boolean; status: "pending" | "approved" | "rejected"; categoryIds: number[]; documents: ProviderDocument[]; // KYC docs rejectionReason: string | null; createdAt: string; commissionRate?: number; photos?: string[]; specialization?: string; brandSpecialization?: string[]; monthlySales?: number; productCategories?: string[]; productsCount?: number; inventoryCount?: number; }
```

3.3 Dealer Dashboard

All endpoints are MISSING. Auth requirement: `provider` role + `providerStatus: "approved"` + provider type `"dealer"`. The `dealerId` is derived from the JWT (no need for the client to send it in the path for self-owned resources).

#	Method	Path	Purpose	Request (key fields)	Response (key fields)	Screen	Status
D1	GET	`/dealer/produc`	List dealer's products (paginated)	`?page, pageSize, status?, search?, categoryId?`	`PaginatedRespc`	■■■■■ ■■■■■■■■ (Products list)	MISSING
D2	GET	`/dealer/produc`	Get single product	—	`Product`	Product detail drawer	MISSING
D3	POST	`/dealer/produc`	Create product	`Product` (omit `id`, `dealerId`, `createdAt`, `updatedAt`)	`Product`	Add product dialog	MISSING
D4	PATCH	`/dealer/produc`	Update product fields	`Partial<Produc`	`Product`	Edit product dialog	MISSING
D5	DELETE	`/dealer/produc`	Delete product	—	`void`	Product list	MISSING
D6	PATCH	`/dealer/produc`	Change product status	`us { status: ProductStatus }`	`Product`	Status toggle in table	MISSING
D7	GET	`/dealer/orders`	List dealer's orders (paginated)	`?page, pageSize, status?, search?`	`PaginatedRespc`	■■■■■■■ (Orders page)	MISSING
D8	GET	`/dealer/orders`	Get single order with items	—	`Order`	Order detail drawer	MISSING
D9	PATCH	`/dealer/orders`	Update order status	`{ status: OrderStatus }`	`Order`	Order status flow	MISSING
D10	POST	`/dealer/orders`	Mark order as shipped, attach tracking	`{ carrier?: string, trackingNumber?: string }`	`Order`	Ship action in order list	MISSING
D11	POST	`/dealer/orders`	Cancel an order	`{ reason?: string }`	`Order`	Cancel action	MISSING
D12	GET	`/dealer/shipme`	List shipments (paginated)	`?page, pageSize, status?`	`PaginatedRespc`	■■■■■■■ > (Shipments page)	MISSING
D13	PATCH	`/dealer/shipme`	Update shipment status	`tu { status: ShipmentStatus }`	`Shipment`	Shipment management	MISSING
D14	GET	`/dealer/dues`	Get dealer financial summary	—	`DealerDues`	■■■■■■■■■ (Dues page)	MISSING

Domain types:

```
type ProductCondition = "new"; type ProductStatus = "active" | "draft" | "out_of_stock" | "archived"; interface Product { id: string; dealerId: string; nameAr: string; nameEn: string; sku: string; categoryId: string; price: number; // SAR condition: ProductCondition; status: ProductStatus; stockQty: number; images?: string[]; descriptionAr?: string;
```

```
descriptionEn?: string; createdAt: string; updatedAt: string; } type OrderStatus = "new" | "preparing" | "shipped" | "delivered" | "cancelled"; interface OrderItem { productId: string; nameAr: string; nameEn: string; sku: string; unitPrice: number; qty: number; lineTotal: number; } interface Order { id: string; dealerId: string; customerName: string; customerPhone?: string; items: OrderItem[]; subtotal: number; commissionRate: number; commissionAmount: number; netToDealer: number; status: OrderStatus; shipmentId?: string; createdAt: string; updatedAt: string; } type ShipmentStatus = "pending" | "in_transit" | "delivered" | "returned"; interface Shipment { id: string; orderId: string; dealerId: string; carrier?: string; trackingNumber?: string; status: ShipmentStatus; shippedAt?: string; deliveredAt?: string; createdAt: string; updatedAt: string; } interface DealerDues { dealerId: string; commissionRate: number; // current % set by admin (read-only to dealer) grossSales: number; // SAR total of delivered orders totalCommission: number; // SAR owed to platform netEarnings: number; // SAR = gross - commission outstandingDebt: number; // SAR current balance debtAlert: boolean; // true if outstandingDebt > 500 updatedAt: string; }
```

3.4 Workshop Dashboard

All endpoints are **MISSING**. Auth: `provider` role + approved + type `"workshop"`. Caller identity derived from JWT (`workshopId` extracted server-side).

#	Method	Path	Purpose	Request (key fields)	Response (key fields)	Screen	Status
W1	GET	`/workshop/pro`	Get workshop's own profile	—	`WorkshopProfi`	■■■■■ ■■■■■■ (Profile page) + Dashboard hero	MISSING
W2	PUT	`/workshop/pro`	Update workshop profile	`Partial<Worksh` (see below)	`WorkshopProfi`	Edit profile form	MISSING
W3	GET	`/workshop/subs`	Get current subscription	—	`WorkshopSubscr`	■■■■■■■■ (Subscription page)	MISSING
W4	POST	`/workshop/subs`	Renew subscription (extends `endDate` by billing cycle)	`ew` —	`WorkshopSubscr`	Renew button on Subscription page	MISSING

Domain types:

```
type DayOfWeek = "sat" | "sun" | "mon" | "tue" | "wed" | "thu" | "fri"; interface DayHours { isClosed: boolean; open?: string; close?: string; } // "HH:mm" type WeeklyHours = Record<DayOfWeek, DayHours>; type WorkshopServiceType = "mechanical" | "bodywork" | "electrical" | "tires" | "periodicMaintenance"; type WorkshopVehicleBrand = "toyota" | "hyundai" | "mercedes" | "ford" | "chevrolet" | "nissan" | "bmw" | "other"; interface WorkshopSpecializations { serviceTypes: WorkshopServiceType[]; vehicleBrands: WorkshopVehicleBrand[]; } interface WorkshopProfile { workshopId: number; companyName: string; email: string; phone: string; address: string | null; city: string | null; workingHoursLabel?: string | null; // display string (legacy, keep for compat) specializationLabel?: string | null; // display string (legacy) bannerUrl?: string; photos: string[]; // gallery URLs location?: { lat: number; lng: number; address: string; city: string }; workingHours: WeeklyHours; // structured schedule specializations: WorkshopSpecializations; contact: { phone: string; whatsapp?: string }; rating: number; totalRatings: number; isVerified: boolean; updatedAt: string; } type WorkshopSubscriptionStatus = "active" | "pending" | "expired" | "cancelled"; type WorkshopBillingCycle = "monthly" | "yearly"; interface WorkshopSubscription { id: string; workshopId: number; planName: string; price: number; billingCycle: WorkshopBillingCycle; status: WorkshopSubscriptionStatus; startDate: string; endDate: string; privileges: { topListing: boolean; freeInspectionOffers: boolean }; renewedAt?: string; }
```

3.5 Scrap Dashboard

All endpoints are **MISSING**. Auth: `provider` role + approved + type `"scrap"`. `scrapId` derived from JWT.

> ■ **ESCROW DOMAIN:** This is entirely new. The backend has no Escrow entity. See Section 6 for the full domain model design requirements.

#	Method	Path	Purpose	Request (key fields)	Response (key fields)	Screen	Status
S1	GET	`/scrap/profile`	Get scrap provider's own profile	—	`ScrapProfile`	■■■■■ ■■■■■■ (Profile page)	MISSING
S2	PUT	`/scrap/profile`	Update profile	`Partial<ScrapP`	`ScrapProfile`	Edit profile form	MISSING
S3	GET	`/scrap/subscri`	Get current subscription	—	`ScrapSubscripti`	■■■■■■■■ (Subscription page)	MISSING
S4	POST	`/scrap/subscri`	Renew subscription	—	`ScrapSubscripti`	Renew button	MISSING
S5	GET	`/scrap/stats`	Dashboard KPI summary	—	`ScrapStats`	■■■■ ■■■■■■ (Dashboard hero cards)	MISSING
S6	GET	`/scrap/part-re`	List part requests visible to this scrap provider	`?page, pageSize, status?, search?`	`PaginatedRespo`	■■■■■ es ■■■■■ (Part Requests page)	MISSING
S7	GET	`/scrap/part-re`	Get single part request detail	—	`PartRequest`	Request detail drawer	MISSING
S8	POST	`/scrap/part-re`	Submit a price offer on a request	`of { price: number, note?: string, photos?: string[] }`	`PartRequest` (updated)	Submit offer dialog	MISSING
S9	PATCH	`/scrap/part-re`	Update part request status (e.g. mark shipped)	`st { status: PartRequestStatus }`	`PartRequest`	Status action in detail	MISSING
S10	GET	`/scrap/escrow/`	Get escrow record for a part request	`Id` —	`Escrow`	Escrow status in request detail	MISSING

Domain types:

```
type ScrapVehicleBrand = "toyota" | "hyundai" | "mercedes" | "ford" | "chevrolet" | "nissan" | "bmw" | "gmc" | "kia" | "other"; interface ScrapProfile { scrapId: number; companyName: string; email: string; bannerUrl?: string; photos: string[]; specializations: { vehicleBrands: ScrapVehicleBrand[] }; location?: { lat: number; lng: number; address: string; city: string }; contact: { phone: string; whatsapp?: string }; rating: number; totalRatings: number; isVerified: boolean; workingHoursLabel?: string | null; updatedAt: string; } type ScrapSubscriptionStatus = "active" | "pending" | "expired" | "cancelled"; type ScrapBillingCycle = "monthly" | "yearly"; interface ScrapSubscription { id: string; scrapId: number; planName: string; price: number; billingCycle: ScrapBillingCycle; status: ScrapSubscriptionStatus; startDate: string; endDate: string; privileges: { priorityListing: boolean; verifiedBadge: boolean }; renewedAt?: string; } type PartRequestStatus = "new" | "quoted" | "accepted" | "shipped" | "completed" | "cancelled"; interface PartRequest { id: string; requestNumber: string; customerName: string; customerPhoneMasked: string; // e.g. "05XXXXX004" — masked by backend vehicle: { brand: ScrapVehicleBrand; model: string; year: number }; partName: string; description?: string; photos: string[]; status: PartRequestStatus; createdAt: string; escrowId?: string; offerPrice?: number; // denormalized from submitted offer offeredAt?: string; } interface ScrapStats { openRequestsCount: number; escrowHeldAmount: number; // SAR currently held in escrow completedDealsCount: number; } type EscrowStatus = "pending" | "held" | "released" | "disputed" | "refunded"; interface Escrow { id: string; partRequestId: string; amount: number; // SAR status: EscrowStatus; heldAt?: string; releasedAt?: string; disputedAt?: string; refundedAt?: string; }
```

Status transition flow (for backend state machine):

```
new → quoted (scrap submits offer) quoted → accepted (customer accepts, escrow created/held) accepted → shipped (scrap marks shipped) shipped → completed (customer confirms receipt, escrow released to scrap) any → cancelled (either party; escrow refunded if held)
```

3.6 Admin Panel

Auth: `admin` or `super_admin` role required for all `/admin/*` endpoints. PARTIAL endpoints are the 3 provider approval actions that already exist.

3.6.1 Dashboard

#	Method	Path	Purpose	Request	Response (key fields)	Screen	Status
AD1	GET	`/admin/dashboa`	Platform KPI overview	—	`DashboardStats`	■■■■ ■■■■■■ ■■■■■■■■	MISSING
AD2	GET	`/admin/revenue`	Revenue records & summary	`?source?: "commission"|`	`RevenueSummary`	■■■■■■■■■ (Revenues page)	MISSING

```
interface DashboardStats { totalUsers: number; activeProviders: number; pendingVerifications: number; monthlyRevenue: number; trends?: { totalUsers: number; activeProviders: number; pendingVerifications: number; monthlyRevenue: number }; revenueSeries?: { month: string; value: number }[]; // last 12 months usersSeries?: { month: string; value: number }[]; providerStatusBreakdown?: { status: string; count: number }[]; } type RevenueSource = "commission" | "subscription"; interface RevenueRecord { id: string; source: RevenueSource; providerId: number; providerName: string; providerType: "dealer"|"workshop"|"scrap"; amount: number; detail?: string; periodMonth?: string; } interface RevenueSummary { totalMonthly: number; commissionTotal: number; subscriptionTotal: number; records: RevenueRecord[]; }
```

3.6.2 Users

#	Method	Path	Purpose	Request	Response	Screen	Status
AU1	GET	`/admin/users`	List all users (paginated)	`?page, pageSize`	`PaginatedResp`	■■■■■ ■■■■■■■■■■	MISSING
AU2	GET	`/admin/users/`	Get user detail	—	`AdminUserDeta`	User detail drawer	MISSING
AU3	POST	`/admin/users/`	Suspend a user account	`{ reason: string }`	`AdminUserRow`	User actions	MISSING
AU4	POST	`/admin/users/`	Restore a suspended user	—	`AdminUserRow`	User actions	MISSING

```
interface AdminUserRow { id: string; name: string; phone: string; email?: string | null; createdAt?: string; role: "customer"|"provider"|"driver"|"admin"|"super_admin"; status: "active"|"suspended"|"pending"; } interface AdminUserDetail extends AdminUserRow { createdAt: string; lastActiveAt: string | null; ordersCount?: number; }
```

3.6.3 Providers (Admin view)

#	Method	Path	Purpose	Request	Response	Screen	Status
AP1	GET	`/admin/provide`	List providers with optional filters	`?status?: "pending"|"a type?: "dealer"|"wo`	`AdminProvider|` (= `ProviderProfi`	■■■■■ ■■■■■ ■■■■■■	PARTIAL (approval only confirmed)
AP2	PATCH	`/admin/provide`	Approve provider KYC	`ov` —	`AdminProvider`	Provider review	EXISTS
AP3	PATCH	`/admin/provide`	Reject with reason	`ct { reason: string }`	`AdminProvider`	Provider review	EXISTS

AP4	PATCH	`/admin/provide`	Set commission rate	`is { commissionRate. number }`	`AdminProvider`	Provider detail	**EXISTS** (per CLAUDE.md)

3.6.4 Services & Categories

#	Method	Path	Purpose	Request	Response	Screen	Status
AS1	GET	`/admin/categor`	List service categories	—	`ServiceCategor`	■■■■■ ■■■■■■■	**MISSING**
AS2	POST	`/admin/categor`	Create category	`{ nameAr, nameEn, iconUrl?, colorHint? }`	`ServiceCategor`	Add category	**MISSING**
AS3	PUT	`/admin/categor`	Update category	`Partial<Servi.`	`ServiceCategor`	Edit category	**MISSING**
AS4	DELETE	`/admin/categor`	Delete category	—	`void`	Category list	**MISSING**
AS5	GET	`/admin/service`	List services	`?categoryId?, isActive?`	`Service[]`	Service list	**MISSING**
AS6	POST	`/admin/service`	Create service	`{ nameAr, nameEn, categoryId, basePrice, estimatedDuration?, isActive, descriptionAr?, descriptionEn?, sortOrder? }`	`Service`	Add service	**MISSING**
AS7	PUT	`/admin/service`	Update service	`Partial<Servi.`	`Service`	Edit service	**MISSING**
AS8	DELETE	`/admin/service`	Delete service	—	`void`	Service list	**MISSING**

3.6.5 Subscription Plans & Provider Subscriptions

#	Method	Path	Purpose	Request	Response	Screen	Status
APL1	GET	`/admin/plans`	List subscription plans	`?isActive?`	`SubscriptionPl`	■■■■■ ■■■■■■■	**MISSING**
APL2	GET	`/admin/plans/:`	Get plan detail	—	`SubscriptionPl`	Plan detail	**MISSING**
APL3	POST	`/admin/plans`	Create plan	`Omit<Subscript "id">`	`SubscriptionPl`	Add plan	**MISSING**
APL4	PUT	`/admin/plans/:`	Update plan	`Partial<Subsc.`	`SubscriptionPl`	Edit plan	**MISSING**
APL5	DELETE	`/admin/plans/:`	Delete plan	—	`void`	Plan list	**MISSING**
APL6	GET	`/admin/subscri`	List active provider subscriptions	`?page, pageSize, status?, type?: "workshop"|"scrap"`	`PaginatedRespo`	■■■■■■■■■■	**MISSING**
APL7	POST	`/admin/subscri`	Cancel provider subscription	`ca —`	`ProviderSubsc.`	Subscription management	**MISSING**

```
interface SubscriptionPlan { id: number; nameAr: string; nameEn: string; descriptionAr?: string | null; descriptionEn?: string | null; price: number; billingCycle: "monthly"|"yearly"; features: { id: string; labelAr: string; labelEn: string }[]; isActive: boolean; sortOrder?: number | null; } interface ProviderSubscription { id: number; providerId: number; providerName: string; providerType: "workshop"|"scrap"; planId: number; planName: string; price: number; billingCycle: "monthly"|"yearly"; status: "active"|"expired"|"cancelled"|"pending"; startDate: string; endDate: string; createdAt: string; }
```

3.6.6 Lookups — Cities, Brands, Models (Admin CRUD)

#	Method	Path	Purpose	Request	Response	Screen	Status
ALK1	GET	`/admin/cities`	List cities	—	`City[]`	■■■■■■■ — ■■■■■	MISSING
ALK2	POST	`/admin/cities`	Create city	`{ nameAr, nameEn }`	`City`	Add city	MISSING
ALK3	PUT	`/admin/cities/`	Update city	`Partial<City>`	`City`	Edit city	MISSING
ALK4	DELETE	`/admin/cities/`	Delete city	—	`void`	City list	MISSING
ALK5	POST	`/admin/brands`	Create vehicle brand	`{ nameAr, nameEn }`	`Brand`	■■■■■ ■■■■■■■■	MISSING
ALK6	PUT	`/admin/brands/`	Update brand	`{ nameAr?, nameEn? }`	`Brand`	Edit brand	MISSING
ALK7	DELETE	`/admin/brands/`	Delete brand	—	`void`	Brand list	MISSING
ALK8	POST	`/admin/brands/`	Add model under brand	`{ nameAr, nameEn }`	`VehicleModel`	Model management	MISSING
ALK9	PUT	`/admin/brands/`	Update model	`{m` `{ nameAr?, nameEn? }`	`VehicleModel`	Edit model	MISSING
ALK10	DELETE	`/admin/brands/`	Delete model	`{m` —	`void`	Model list	MISSING

Note: `GET /Lookups/brands`, `GET /Lookups/brands/{id}/models`, `GET /Lookups/brands/{id}/models/{modelId}/years` already **EXIST** for reads. Admin CRUD is separate.

3.6.7 Notifications

#	Method	Path	Purpose	Request	Response	Screen	Status
AN1	GET	`/admin/notifi`	List templates	`te` `?isActive?`	`NotificationTe`	■■■■■ ■■■■■■■■■	MISSING
AN2	GET	`/admin/notifi`	Get template	`te` —	`NotificationTe`	Template detail	MISSING
AN3	POST	`/admin/notifi`	Create template	`te` `Omit<Notificat "id">`	`NotificationTe`	Add template	MISSING
AN4	PUT	`/admin/notifi`	Update template	`te` `Partial<Notif:`	`NotificationTe`	Edit template	MISSING
AN5	DELETE	`/admin/notifi`	Delete template	`te` —	`void`	Template list	MISSING
AN6	POST	`/admin/notifi`	Send a notification to audience	`{ templateId?, titleAr, titleEn, bodyAr, bodyEn, audience, channel }`	`SentNotificati`	■■■■■ ■■■■■	MISSING

AN7	GET	`/admin/notifi`	List sent notifications	`?page, pageSize, status?`	`PaginatedResp`	■■■ ■■■■■■■■■	MISSING

```
type NotificationChannel = "in_app" | "push" | "email" | "sms"; type NotificationAudience = "all" | "customers" | "providers"; type NotificationStatus = "pending" | "sent" | "failed"; interface NotificationTemplate { id: string; nameAr: string; nameEn: string; titleAr: string; titleEn: string; bodyAr: string; bodyEn: string; variables: string[]; // template variable names e.g. ["customerName"] channel: NotificationChannel; isActive: boolean; } interface SentNotification { id: string; templateId?: string | null; titleAr: string; titleEn: string; bodyAr: string; bodyEn: string; audience: NotificationAudience; channel: NotificationChannel; status: NotificationStatus; sentAt: string; recipientsCount: number; }
```

3.6.8 Ads (Campaigns & Placements)

#	Method	Path	Purpose	Request	Response	Screen	Status
AAD1	GET	`/admin/ad-pla`	List ad placements/slots	`?isActive?`	`AdPlacement[]`	■■■■■ ■■■■■■■■■	MISSING
AAD2	POST	`/admin/ad-pla`	Create placement	`Omit<AdPlaceme "id">`	`AdPlacement`	Add placement	MISSING
AAD3	PUT	`/admin/ad-pla`	Update placement	`Partial<Omit< "id">>`	`AdPlacement`	Edit placement	MISSING
AAD4	DELETE	`/admin/ad-pla`	Delete placement	—	`void`	Placement list	MISSING
AAD5	GET	`/admin/ad-cam`	List campaigns (paginated)	`?page, pageSize, status?, placementId?`	`PaginatedResp`	■■■■■■■ ■■■■■■■■■	MISSING
AAD6	GET	`/admin/ad-cam`	Get campaign detail	—	`AdCampaign`	Campaign detail	MISSING
AAD7	POST	`/admin/ad-cam`	Create campaign	`Omit<AdCampai "id"|"created`	`AdCampaign`	Add campaign	MISSING
AAD8	PUT	`/admin/ad-cam`	Update campaign	`Partial<Omit< "id"|"created`	`AdCampaign`	Edit campaign	MISSING
AAD9	DELETE	`/admin/ad-cam`	Delete campaign	—	`void`	Campaign list	MISSING

```
type AdCampaignStatus = "draft" | "scheduled" | "active" | "paused" | "ended"; interface AdPlacement { id: number; code: string; nameAr: string; nameEn: string; descriptionAr?: string; descriptionEn?: string; isActive: boolean; } interface AdCampaign { id: number; titleAr: string; titleEn: string; descriptionAr?: string; descriptionEn?: string; imageUrl?: string; targetUrl?: string; placementId: number; startsAt: string; endsAt: string; status: AdCampaignStatus; priority?: number; createdAt: string; }
```

3.6.9 System Settings

#	Method	Path	Purpose	Request	Response	Screen	Status
AST1	GET	`/admin/settin`	Get all system settings	—	`SystemSettings`	■■■■■■■ ■■■■■■	MISSING
AST2	PUT	`/admin/settin`	Update general settings	`GeneralSettin`	`SystemSettings`	General tab	MISSING
AST3	PUT	`/admin/settin`	Update contact/legal URLs	`ContactSettin`	`SystemSettings`	Contact tab	MISSING
AST4	PUT	`/admin/settin`	Update feature flags	`gs FeatureFlag[]`	`SystemSettings`	Feature flags tab	MISSING

```
interface SystemSettings { general: { platformNameAr: string; platformNameEn: string; logoUrl?: string; supportEmail: string; supportPhone: string; defaultCurrency: string; defaultLanguage: "ar"|"en"; timezone: string; }; contact: { termsUrlAr?: string; termsUrlEn?: string; privacyUrlAr?: string; privacyUrlEn?: string; twitterUrl?: string; instagramUrl?: string; whatsappNumber?: string; }; featureFlags: { key: string; labelAr: string; labelEn: string; descriptionAr?: string; descriptionEn?: string; enabled: boolean }[]; }
```

3.6.10 Audit Logs

#	Method	Path	Purpose	Request	Response	Screen	Status
AAL1	GET	`/admin/audit-`	Query audit log (paginated)	`?page, pageSize, module?, action?, actorId?, dateFrom?, dateTo?, search?`	`PaginatedRespo`	■■■ ■■■■■■■	En MISSING
AAL2	GET	`/admin/audit-`	Export audit log as file	`?module?, action?, actorId?, dateFrom?, dateTo?, search?`	`Blob` (CSV or Excel)	Export button	MISSING

```
type AuditAction = "create"|"update"|"delete"|"approve"|"reject"|"suspend"|"restore"|"send"|"cancel"|"login"; type AuditModule = "users"|"providers"|"services"|"plans"|"subscriptions"|"notifications"|"ads"|"settings"|"auth"; interface AuditLogEntry { id: number; actorId: string; actorName: string; actorRole: string; action: AuditAction; module: AuditModule; entityType?: string; entityId?: string; summaryAr: string; summaryEn: string; metadata?: Record<string, unknown>; ipAddress?: string; createdAt: string; }
```

3.6.11 Complaints

#	Method	Path	Purpose	Request	Response	Screen	Status
ACM1	GET	`/admin/compla`	List complaints (paginated)	`?page, pageSize, status?, search?`	`PaginatedRespo`	■■■■■■■	t> MISSING
ACM2	PATCH	`/admin/compla`	Update complaint status	tu `{ status: ComplaintStatus }`	`Complaint`	Complaint detail	MISSING

```
type ComplaintStatus = "new" | "under_review" | "resolved"; interface Complaint { id: string; customerName: string; title: string; body: string; status: ComplaintStatus; createdAt: string; }
```

3.7 Discovery (Customer-Facing — currently unimplemented)

#	Method	Path	Purpose	Request	Response	Screen	Status
DC1	GET	`/services/near`	Search providers by location + filters	`?lat, lng, radiusKm, categoryId?, minRating?, priceBand?, openNowOnly?, q?`	`NearbyProvider`	■■■ ■■ ■■■■■ ■■■■■■	MISSING
DC2	GET	`/ServiceProvi`	Get provider reviews	ie `?limit?, offset?`	`{ total, averageRating, items: ProviderReview[] }`	Provider profile page	MISSING

DC3	GET	`/favorites`	List current user's favorites	—	`{ providerId: number, createdAt: string }[]`	■■■■■■■	MISSING
DC4	POST	`/favorites/{pı`	Add to favorites	—	`{ providerId, createdAt }`	Provider card / profile	MISSING
DC5	DELETE	`/favorites/{pı`	Remove from favorites	—	`{ success: true }`	Provider card / profile	MISSING

`priceBand` values: `"lte_100" | "100_300" | "300_700" | "gt_700"`

3.8 Services / Lookups (Customer-Facing Reads)

#	Method	Path	Purpose	Request	Response	Screen	Status
SL1	GET	`/Services/cat:`	List service categories	—	`ServiceCategor`	Service picker; discovery filters	PARTIAL (check Swagger)
SL2	GET	`/Services/cat:`	List `su` subcategories	—	`ServiceSubcat:`	Service picker	PARTIAL
SL3	GET	`/Lookups/bran`	List vehicle brands	—	`Brand[]`	Vehicle add/edit form	EXISTS
SL4	GET	`/Lookups/bran`	List models for brand `s`	—	`VehicleModel[]`	Vehicle form	EXISTS
SL5	GET	`/Lookups/bran`	List years for model `s/`	— `ar`	`number[]`	Vehicle form	EXISTS

---

4. Missing-Endpoint Checklist

Copy this into your task tracker. Each line is one backend endpoint to build.

AUTH / ONBOARDING

- [ ] `GET /onboarding/account-summary` — return summary of newly-registered account for onboarding wizard
- [ ] `POST /onboarding/documents` — upload a single KYC doc during onboarding; return `{ fileId, fileName }`
- [ ] `POST /onboarding/submit-review` — submit provider application for admin review; return `{ submittedAt }`
- [ ] `GET /onboarding/review-status` — poll review progress; return timeline items with states

PROVIDER

- [ ] `GET /provider/me` — return the calling provider's own `ProviderProfile` (derived from JWT, no ID in path)

DEALER DASHBOARD

- [ ] `GET /dealer/products` — paginated product list (`?page, pageSize, status?, search?, categoryId?`)
- [ ] `GET /dealer/products/{id}` — get single product
- [ ] `POST /dealer/products` — create product
- [ ] `PATCH /dealer/products/{id}` — update product fields
- [ ] `DELETE /dealer/products/{id}` — delete product

- [ ] `PATCH /dealer/products/{id}/status` — change product status only; body `{ status }`
- [ ] `GET /dealer/orders` — paginated order list (`?page, pageSize, status?, search?`)
- [ ] `GET /dealer/orders/{id}` — get single order with `items[]`
- [ ] `PATCH /dealer/orders/{id}/status` — update order status; body `{ status }`
- [ ] `POST /dealer/orders/{id}/ship` — mark shipped + attach carrier/tracking; body `{ carrier?, trackingNumber? }`
- [ ] `POST /dealer/orders/{id}/cancel` — cancel order; body `{ reason? }`
- [ ] `GET /dealer/shipments` — paginated shipment list (`?page, pageSize, status?`)
- [ ] `PATCH /dealer/shipments/{id}/status` — update shipment status; body `{ status }`
- [ ] `GET /dealer/dues` — dealer financial summary (commissions, net earnings, debt alert)

WORKSHOP DASHBOARD

- [ ] `GET /workshop/profile` — get workshop's own profile
- [ ] `PUT /workshop/profile` — update profile (banner, photos, location, hours, specializations, contact)
- [ ] `GET /workshop/subscription` — get current subscription with privileges
- [ ] `POST /workshop/subscription/renew` — extend subscription by one billing cycle

SCRAP DASHBOARD

- [ ] `GET /scrap/profile` — get scrap provider's own profile
- [ ] `PUT /scrap/profile` — update profile
- [ ] `GET /scrap/subscription` — get current subscription
- [ ] `POST /scrap/subscription/renew` — renew subscription
- [ ] `GET /scrap/stats` — dashboard KPIs: `{ openRequestsCount, escrowHeldAmount, completedDealsCount }`
- [ ] `GET /scrap/part-requests` — paginated part request list (`?page, pageSize, status?, search?`)
- [ ] `GET /scrap/part-requests/{id}` — single part request detail
- [ ] `POST /scrap/part-requests/{id}/offer` — submit offer; body `{ price, note?, photos? }`
- [ ] `PATCH /scrap/part-requests/{id}/status` — update status; body `{ status }`
- [ ] `GET /scrap/escrow/{partRequestId}` — get escrow state for a part request

ADMIN — DASHBOARD & REVENUES

- [ ] `GET /admin/dashboard/stats` — platform KPIs with trend deltas and chart series
- [ ] `GET /admin/revenues` — revenue records summary (`?source?: "commission"|"subscription"`)

ADMIN — USERS

- [ ] `GET /admin/users` — paginated user list (`?page, pageSize`)
- [ ] `GET /admin/users/{id}` — user detail with activity stats
- [ ] `POST /admin/users/{id}/suspend` — suspend user; body `{ reason }`
- [ ] `POST /admin/users/{id}/restore` — restore suspended user

ADMIN — PROVIDERS

- [ ] `GET /admin/providers` — list providers with optional filters (`?status?, type?`)
- [ ] `PATCH /admin/providers/{id}/commission` — set commission rate; body `{ commissionRate }`

ADMIN — SERVICES & CATEGORIES

- [ ] `GET /admin/categories` — list service categories
- [ ] `POST /admin/categories` — create category
- [ ] `PUT /admin/categories/{id}` — update category
- [ ] `DELETE /admin/categories/{id}` — delete category
- [ ] `GET /admin/services` — list services (`?categoryId?, isActive?`)
- [ ] `POST /admin/services` — create service
- [ ] `PUT /admin/services/{id}` — update service
- [ ] `DELETE /admin/services/{id}` — delete service

ADMIN — SUBSCRIPTION PLANS

- [ ] `GET /admin/plans` — list subscription plans (`?isActive?`)
- [ ] `GET /admin/plans/{id}` — get single plan
- [ ] `POST /admin/plans` — create plan
- [ ] `PUT /admin/plans/{id}` — update plan
- [ ] `DELETE /admin/plans/{id}` — delete plan
- [ ] `GET /admin/subscriptions` — list provider subscriptions (`?page, pageSize, status?, type?`)
- [ ] `POST /admin/subscriptions/{id}/cancel` — cancel a provider subscription

ADMIN — LOOKUPS (CRUD)

- [ ] `GET /admin/cities` — list cities
- [ ] `POST /admin/cities` — create city
- [ ] `PUT /admin/cities/{id}` — update city
- [ ] `DELETE /admin/cities/{id}` — delete city
- [ ] `POST /admin/brands` — create vehicle brand
- [ ] `PUT /admin/brands/{id}` — update brand
- [ ] `DELETE /admin/brands/{id}` — delete brand
- [ ] `POST /admin/brands/{id}/models` — add model under brand
- [ ] `PUT /admin/brands/{id}/models/{modelId}` — update model
- [ ] `DELETE /admin/brands/{id}/models/{modelId}` — delete model

ADMIN — NOTIFICATIONS

- [ ] `GET /admin/notification-templates` — list templates (`?isActive?`)
- [ ] `GET /admin/notification-templates/{id}` — get template
- [ ] `POST /admin/notification-templates` — create template
- [ ] `PUT /admin/notification-templates/{id}` — update template
- [ ] `DELETE /admin/notification-templates/{id}` — delete template
- [ ] `POST /admin/notifications/send` — send a notification broadcast
- [ ] `GET /admin/notifications` — list sent notifications (`?page, pageSize, status?`)

ADMIN — ADS

- [ ] `GET /admin/ad-placements` — list ad slots (`?isActive?`)
- [ ] `POST /admin/ad-placements` — create placement
- [ ] `PUT /admin/ad-placements/{id}` — update placement
- [ ] `DELETE /admin/ad-placements/{id}` — delete placement
- [ ] `GET /admin/ad-campaigns` — paginated campaigns (`?page, pageSize, status?, placementId?`)
- [ ] `GET /admin/ad-campaigns/{id}` — get campaign
- [ ] `POST /admin/ad-campaigns` — create campaign
- [ ] `PUT /admin/ad-campaigns/{id}` — update campaign
- [ ] `DELETE /admin/ad-campaigns/{id}` — delete campaign

ADMIN — SETTINGS

- [ ] `GET /admin/settings` — get all system settings
- [ ] `PUT /admin/settings/general` — update general settings section
- [ ] `PUT /admin/settings/contact` — update contact/legal URLs section
- [ ] `PUT /admin/settings/featureFlags` — update feature flags section

ADMIN — AUDIT LOGS

- [ ] `GET /admin/audit-logs` — paginated audit log (`?page, pageSize, module?, action?, actorId?, dateFrom?, dateTo?, search?`)
- [ ] `GET /admin/audit-logs/export` — export filtered log as file (CSV / Excel)

ADMIN — COMPLAINTS

- [ ] `GET /admin/complaints` — paginated complaints (`?page, pageSize, status?, search?`)
- [ ] `PATCH /admin/complaints/{id}/status` — update status; body `{ status }`

DISCOVERY (Customer)

- [ ] `GET /services/nearby` — nearby provider search (`?lat, lng, radiusKm, categoryId?, minRating?, priceBand?, openNowOnly?, q?`)
- [ ] `GET /ServiceProviders/{id}/reviews` — provider reviews (`?limit?, offset?`)
- [ ] `GET /favorites` — current user's favorite providers
- [ ] `POST /favorites/{providerId}` — add to favorites
- [ ] `DELETE /favorites/{providerId}` — remove from favorites

SERVICES / LOOKUPS (if not already present)

- [ ] `GET /Services/categories` — list service categories (verify in Swagger)
- [ ] `GET /Services/categories/{id}/subcategories` — list subcategories

---

5. Domain Models To Design (Net-New Backend Entities)

The following entities are assumed by the frontend but do not exist in the current backend. The backend team must design storage, migrations, and business logic for each.

5.1 PartRequest

Customer-submitted request for a scrap/salvage part. Visible to all scrap providers in their region.

Field	Type	Notes
`id`	`string` (UUID)	PK
`requestNumber`	`string`	Human-read reference e.g. "PR-2026-0
`customerName`	`string`	
`customerPhoneMasked`	`string`	Masked by backend e.g. "05XXXXX0
`vehicle.brand`	`string` enum	e.g. `"toyota"`
`vehicle.model`	`string`	
`vehicle.year`	`number`	
`partName`	`string`	
`description`	`string?`	
`photos`	`string[]`	URLs
`status`	`"new"|"quoted"|"accepted"|"shipped"|"completed"|"ca`	State machine
`createdAt`	`string`	ISO-8601
`escrowId`	`string?`	FK to Escrow — set when status moves to "accepted"
`offerPrice`	`number?`	Denormalize from the scrap provider's offer
`offeredAt`	`string?`	ISO-8601

5.2 Escrow

Payment held by the platform between a customer's acceptance and delivery confirmation. **This is a new financial domain — no existing backend analogue.**

Field	Type	Notes
`id`	`string` (UUID)	PK

`partRequestId`	`string`	FK to PartReques
`amount`	`number`	SAR — matches the accepted offer price
`status`	`"pending"|"held"|"released"|"disputed"|"refunded"`	
`heldAt`	`string?`	ISO when payment was captured
`releasedAt`	`string?`	ISO when released to scrap provider
`disputedAt`	`string?`	ISO when dispute was opened
`refundedAt`	`string?`	ISO when refunded to customer

Business rule: Escrow transitions are buyer-driven (customer confirms receipt → released) or admin-arbitrated (dispute). The scrap provider cannot release their own escrow.

5.3 Product (Dealer)

Field	Type	Notes
`id`	`string` (UUID)	PK
`dealerId`	`string`	FK to Provider (type=deale
`nameAr` / `nameEn`	`string`	Bilingual
`sku`	`string`	Provider-ass
`categoryId`	`string`	FK to service/prod category
`price`	`number`	SAR
`condition`	`"new"`	Extensible
`status`	`"active"|"draft"|"out_of_stock"|"archived"`	

`stockQty`	`number`	Denormalized source of truth = Inventory
`images`	`string[]`	URLs
`descriptionAr` / `descriptionEn`	`string?`	
`createdAt` / `updatedAt`	`string`	ISO-8601

5.4 Inventory (Dealer)

Field	Type	Notes
`productId`	`string`	FK to Product
`dealerId`	`string`	FK to Provider
`onHand`	`number`	Current physical stock
`reserved`	`number`	Reserved by open orders
`updatedAt`	`string`	ISO-8601

5.5 Order (Dealer)

Full structure documented in Section 3.3. Key design notes: - `commissionRate` is a **snapshot** taken at order creation time — immutable thereafter - `commissionAmount = subtotal * commissionRate / 100` - `netToDealer = subtotal - commissionAmount` - `customerPhone` is displayed to dealer but may warrant masking policy

5.6 Shipment (Dealer)

Full structure in Section 3.3. Linked 1-to-1 with an Order via `orderId`. Created when `POST /dealer/orders/{id}/ship` is called.

5.7 DealerDues

Aggregated financial view — not a stored record but a computed response: - `grossSales` = sum of `subtotal` for all orders with `status: "delivered"` - `totalCommission` = sum of `commissionAmount` for delivered orders - `netEarnings = grossSales - totalCommission` - `outstandingDebt` = running balance of unpaid commission (business rule TBD — monthly settlement?) - `debtAlert = outstandingDebt > 500`

5.8 ProviderSubscription (Workshop & Scrap)

Both workshop and scrap have their own subscription shape (see Sections 3.4 and 3.5). The admin-facing `ProviderSubscription` (Section 3.6.5) is the unified admin view. The provider-facing schemas differ in `privileges` field: - Workshop: `{ topListing: boolean, freeInspectionOffers: boolean }` - Scrap: `{ priorityListing: boolean, verifiedBadge: boolean }`

These can either be stored as a JSON column or normalized into a `SubscriptionPrivilege` table.

5.9 RevenueRecord (Admin)

Computed from orders (commission) and subscription payments. The backend needs logic to aggregate monthly: - Commission revenue = sum of `commissionAmount` per `dealerId` per month - Subscription revenue = subscription `price` per active `ProviderSubscription` per month

5.10 AdPlacement + AdCampaign

New tables; no existing backend analogue. `AdPlacement` defines display slots (e.g., "home-banner", "search-sidebar"). `AdCampaign` links content + schedule to a placement.

5.11 AuditLog

System-generated on every mutating admin action. The backend must emit an audit entry on every write to: users, providers, services, plans, subscriptions, notifications, ads, settings. Fields: `actorId`, `action`, `module`, `entityType`, `entityId`, `summaryAr`, `summaryEn`, `metadata`, `ipAddress`.

5.12 Complaint

Customer-submitted complaint visible in admin panel. Source of submission not specified in frontend (likely a customer-facing form endpoint separate from this dashboard spec).

5.13 NotificationTemplate + SentNotification

Template defines bilingual content + channel + variable placeholders. SentNotification records a broadcast event with delivery status and recipient count.

5.14 WorkshopProfile & ScrapProfile

Both are **extended provider profiles** layered on top of the base `ProviderProfile`. Design choices: 1. **Polymorphic (recommended):** Store all fields on the `ServiceProvider` table with nullable columns per type, plus a JSONB column for structured data (`workingHours`, `specializations`, `contact`, `location`) 2. **Separate tables:** `WorkshopProfile`, `ScrapProfile` each with FK to `ServiceProvider.id` — cleaner normalization but requires joins

---

6. Wiring Priority

Tier 1 — Auth & Identity (unblocks everything)

1. OTP-based auth (`/auth/register`, `/auth/verify-otp`) — reconcile with existing backend
2. `/users/me` GET/PUT + avatar upload
3. `/provider/me` — provider dashboard bootstrap
4. `/onboarding/*` — 4 endpoints

Tier 2 — Admin Panel (parallel-buildable with Tier 3)

5. `/admin/dashboard/stats` — unlocks admin home page
6. `/admin/providers` list + commission PATCH
7. `/admin/users` CRUD (4 endpoints)
8. `/admin/plans` CRUD (5 endpoints) — needed before workshop/scrap subscriptions
9. `/admin/subscriptions` (2 endpoints)
10. `/admin/revenues` (1 endpoint)
11. `/admin/categories` + `/admin/services` CRUD (8 endpoints)
12. `/admin/cities` CRUD + `/admin/brands` + `/admin/brands/{id}/models` CRUD (10 endpoints)
13. `/admin/notification-templates` CRUD + `/admin/notifications/send` + `/admin/notifications` (7 endpoints)
14. `/admin/ad-placements` + `/admin/ad-campaigns` CRUD (9 endpoints)
15. `/admin/settings` GET/PUT (4 endpoints)
16. `/admin/audit-logs` + export (2 endpoints)
17. `/admin/complaints` (2 endpoints)

Tier 3 — Provider Dashboards

18. Dealer: `/dealer/products` CRUD (6) → `/dealer/orders` + status actions (5) → `/dealer/shipments` (2) → `/dealer/dues` (1)
19. Workshop: `/workshop/profile` GET/PUT + `/workshop/subscription` GET/renew (4 endpoints)
20. Scrap: `/scrap/profile` GET/PUT + `/scrap/subscription` GET/renew + `/scrap/stats` (5 endpoints) → `/scrap/part-requests` CRUD + offer + status (5 endpoints) → `/scrap/escrow` (1 endpoint) — **design Escrow domain first**

Tier 4 — Discovery (Customer App)

21. `/services/nearby` + `/ServiceProviders/{id}/reviews` + `/favorites` CRUD (5 endpoints)

---

7. Open Questions for Backend Team

#	Question	Why it blocks
OQ1	**OTP vs Password:** Does the current backend support OTP login (`/auth/register` + `/auth/verify-otp`)? If only password login exists, this must be added before any provider/admin frontend can be wired.	All authenticated dashboards
OQ2	**Response envelope:** Does the backend emit bare responses or `{ success, data }` wrapped? The frontend currently consumes bare `T` for single resources and `PaginatedResponse<T>` for lists. If the backend wraps, an Axios interceptor must be added.	All endpoints
OQ3	**Field casing:** Will the backend emit camelCase JSON (`JsonNamingPolicy.CamelCase`)? If PascalCase, a transform layer is needed on the frontend.	All endpoints
OQ4	**Provider type namespaces vs polymorphic:** Should dealer/workshop/scrap use separate endpoint prefixes (`/dealer/*`, `/workshop/*`, `/scrap/*`) as the frontend expects, or a single polymorphic `/provider/*` with type discrimination? Frontend strongly expects separate namespaces.	Dealer, Workshop, Scrap dashboards
OQ5	**Escrow ownership:** Who holds escrow funds — a platform wallet, a third-party payment gateway, or a ledger entry? The frontend assumes release is triggered by customer confirmation; does the backend have a payment gateway integration that supports holds?	Scrap Escrow domain (S10)
OQ6	**`providerId` type:** The frontend uses `id: number` for providers (matching the existing `ServiceProviders/{id}` integer PK) but `id: string` (UUID) for new entities (products, orders). Is the backend migrating provider IDs to UUIDs or keeping integers?	All provider-scoped endpoints

OQ7	**Admin `GET /admin/providers` — is this the same as the existing approval endpoint?** The frontend calls this with optional `status` and `type` filters to list all providers, not just pending ones. Confirm whether the existing approval endpoint can be extended or a new one is needed.	Admin provider list
OQ8	**Dealer `commissionRate` at order time:** The frontend snapshots `commissionRate` into each `Order` at creation. Does the backend store this on the order row, or compute it dynamically? The `DealerDues` computation depends on this snapshot being immutable.	Dealer orders and dues
OQ9	**`customerPhoneMasked` in PartRequest:** The backend must mask the customer phone before returning it to the scrap provider. What is the masking format — `05XXXXX004` (last 3 digits visible)? Is this a backend-computed field or a frontend concern?	Scrap part requests
OQ10	**OutstandingDebt settlement cycle in `DealerDues`:** What business rule defines when `outstandingDebt` resets — monthly invoice cycle, per-order settlement, or manual admin clearing? The `debtAlert` threshold (500 SAR) is defined in the SRS (FR-MKT-07).	Dealer dues endpoint